

Решение практических задач при помощи библиотек Python

Установка сторонних модулей

Существует специальный сайт (репозиторий) со списком сторонних пакетов

<https://pypi.org/> 766 358 число пакетов на сегодняшний день

Модуль **pip** отвечает за установку сторонних пакетов в интерпретатор.

`pip install pygame`

Чтобы увидеть, какие пакеты уже установлены для текущего интерпретатора Python, можно набрать команду:

`pip list`

Пакетная установка позволяет установить список пакетов заданной версии из файла

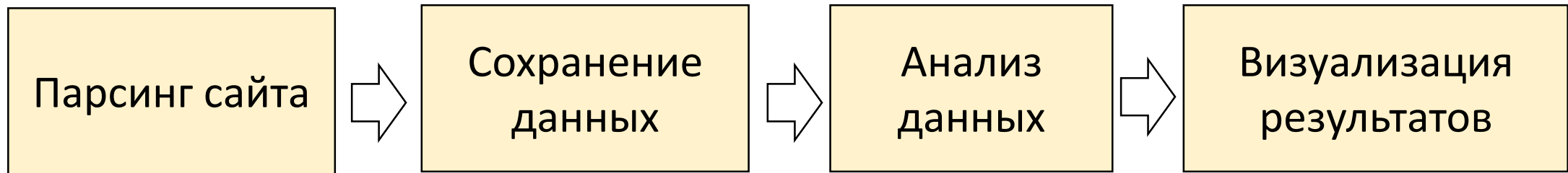
`pip install -r <текстовый файл>`



```
numpy==1.19.0
Pillow==7.2.0
six==1.15.0
wxPython==4.1.0
```

Задача:

Проанализировать цены на товары у конкурентов



Определить тональность отзывов на товары
Приложение для анализа курса валют . . .

Парсинг — это автоматизированный сбор общедоступной информации из интернета.

Можно представить, как человек открывает браузер, ходит по сайтам и копирует с них данные. Парсинг — то же самое, только ходит не человек, а робот. Так делают поисковики, агрегаторы, скоринговые компании, злоумышленники, продающие персональные данные, и много кто ещё.

Парсинг подходит для быстрого сбора большого объема данных.

Парсинг — не совсем точное название. Автоматизированный сбор информации лучше называть скрейпингом, а парсинг — это этап скрейпинга, на котором из скачанных данных извлекается нужная информация. Но в русскоязычных источниках чаще попадаетея слово «парсинг»

Взаимодействие клиента и сервера строится по модели «запрос-ответ».

Запрос клиента: Пользователь совершает действие (открывает сайт, нажимает кнопку), и клиент формирует запрос (**GET, POST** и т.д.) с данными или командой.

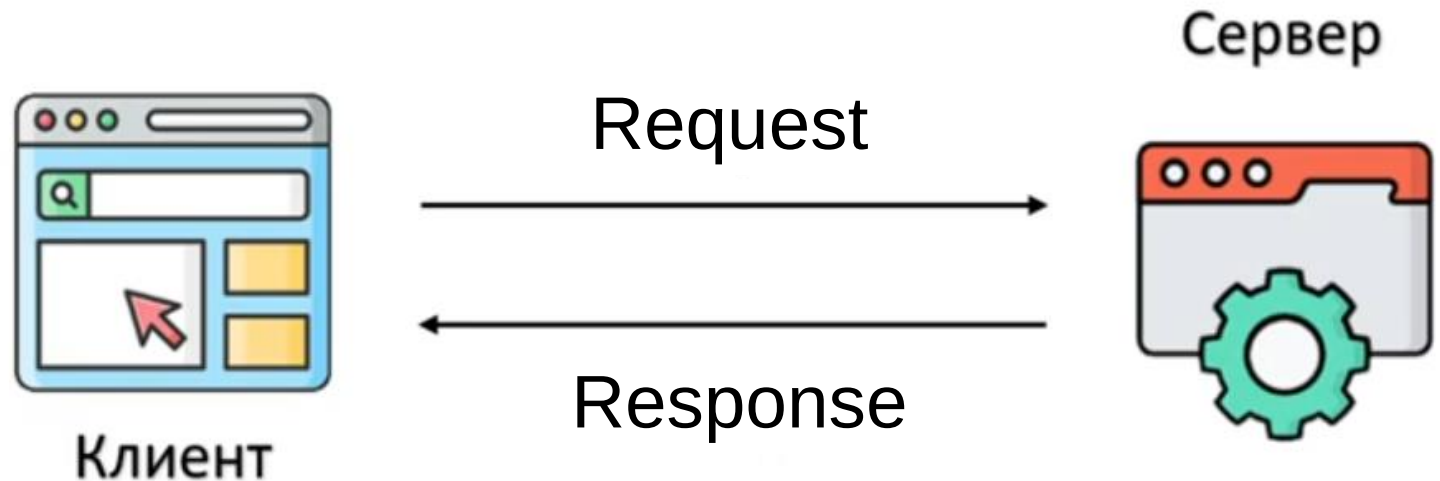
Отправка запроса: Запрос передается по сети (используя протокол HTTP/HTTPS).

Прием и обработка сервером: Сервер получает запрос, проверяет его подлинность, обрабатывает логику (обращается к БД, выполняет вычисления).

Ответ сервера: Сервер формирует ответ (данные, HTML-страница, код ошибки) и отправляет его обратно.

Получение ответа клиентом:

Клиент получает данные и отображает их пользователю.



Парсинг. Шаг 1: Получаем HTML-страницу

Requests - это модуль Python, который используется для отправки всех видов HTTP-запросов.

Requests не используется как самостоятельный парсер-скрипт, но она является важной составляющей при работе со структурами URL-адресов и при получении данных.

! Библиотека Requests не работает со страницами, написанными на JavaScript.

Пример: С помощью модуля requests получим страницу для парсинга. Для этого нужно отправить GET-запрос, а в качестве ответа получить код страницы и сохранить его:

```
import requests  
url = 'https://belstu.by/'  
data = requests.get(url)  
print(data)  
print(data.text)
```

import requests — импортируем библиотеку requests в код скрипта;

url — переменная, в которой хранится ссылка на целевую страницу;

data — выполняем GET-запрос и передаем в него переменную с хранящейся ссылкой;

print(data) — напечатает код состояния ответа;

print(response.text) — печатаем код полученной страницы.

Парсинг. Шаг 2: Парсим страницу

Извлечь полезные данные из полученной html-страницы можно с помощью множества библиотек для парсинга, самые популярные из них:



Beautiful Soup (BS4) –библиотека для парсинга html и xml документов. Позволяет получить доступ напрямую к содержимому любых тегов в html.



Selenium – это набор инструментов и утилит автоматизации, обеспечивающих взаимодействие с движками браузеров. Они необходимы для поддержки некоторых важных технологий, таких как JavaScript и асинхронный AJAX. Selenium применяют для парсинга динамически загружаемых страниц.



Scrapy – это Python-фреймворк, обеспечивающий быстрое создание асинхронных парсеров данных с веб-сайтов.

Парсинг при помощи BeautifulSoup

Для работы нужно установить библиотеку:

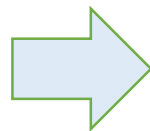
```
pip install beautifulsoup4
```

Beautiful Soup представляет html-документ в виде вложенной структуры данных, объект `soup`:

```
html_doc = """
<html><head><title>The Dormouse's st
<body>
<p class="title"><b>The Dormouse's s

<p class="story">Once upon a time th
<a href="http://example.com/elsie" c
<a href="http://example.com/lacie" c
<a href="http://example.com/tillie"
and they lived at the bottom of a we

<p class="story">...</p>
"""
```



```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html_doc, 'html.parser')

print(soup.prettify())
# <html>
#   <head>
#     <title>
#       The Dormouse's story
#     </title>
#   </head>
#   <body>
#     <p class="title">
#       <b>
#         The Dormouse's story
#       </b>
#     </p>
#     <p class="story">
#       Once upon a time there were three little sister
#       <a class="sister" href="http://example.com/elsi
#         Elsie
#       </a>
#
```

Библиотека будет использовать HTML-парсер, если явно не указать, что нужно использовать XML-парсер.

Beautiful Soup. Поиск элементов в объекте soup

См [документацию](#)

Beautiful Soup определяет множество методов поиска по дереву разбора, самые популярные методы: **find()** и **find_all()**. В эти методы можно передавать различные фильтры, строки, регулярные выражения, функции и т.д.

Строка. *Найдем все теги `<b>` в документе:*

```
soup.find_all('b')
```

Список. *Следующий код находит все теги `<a>` и все теги `<b>`:*

```
soup.find_all(["a", "b"])
```

Регулярное выражение. *Найдем все теги, имена которых начинаются с буквы "b":*

```
import re
for tag in soup.find_all(re.compile("^b")):
    print(tag.name)
```

Функция. *Вот функция, которая возвращает True, если в теге определен атрибут "class", но не определен атрибут "id":*

```
def has_class_but_no_id(tag):
```

```
    return tag.has_attr('class') and not tag.has_attr('id')
```

```
soup.find_all(has_class_but_no_id) Передайте эту функцию в find_all(), и получите все теги <p>
```

```
import requests  
from bs4 import BeautifulSoup
```

Пример работы с
библиотекой
BeautifulSoup

```
# GET запрос к сайту с целью получения html кода  
html = requests.get('https://sunlight.net/catalog').text
```

```
# создаём экземпляр объекта BeautifulSoup в который передаём переменную  
html.
```

```
soup = BeautifulSoup(html, 'lxml')
```

```
# получаем текст конкретного заголовка  
str=soup.find('h2', class_="cl-reviews__title").text  
print(str)
```

```
# получаем список элементов  
items = soup.find_all('div', class_='cl-item-mobile-footer')  
print(len(items))
```

Парсинг при помощи Selenium

Установка `pip install selenium`

Selenium предоставляет следующие методы поиска элементов на странице:

`find_element_by_id`

`find_element_by_name`

`find_element_by_xpath`

`find_element_by_link_text`

`find_element_by_partial_link_text`

`find_element_by_tag_name`

`find_element_by_class_name`

`find_element_by_css_selector`

Для поиска всех элементов, удовлетворяющих условию поиска, следующие методы (возвращается список):

`find_elements_by_name`

`find_elements_by_xpath`

`find_elements_by_link_text`

`find_elements_by_partial_link_text`

`find_elements_by_tag_name`

`find_elements_by_class_name`

`find_elements_by_css_selector`

Selenium. Настройка ожиданий

Так как скрипт может отработать быстрее, чем динамически обновится страница и загрузятся все элементы, необходимо пользоваться ожиданиями.

- ✓ Самое простое решение, но не самое лучшее использование метода `time.sleep()` из модуля `time`:

```
import time
...
time.sleep(3)
```

- ✓ Неявные ожидания - Implicit Waits :

В данном примере наличие элемента будет проверяться каждые 500 мс. Если элемент найден раньше, сразу перейдем к следующему шагу.

```
driver.implicitly_wait(5)
driver.get("https://example.com")
element = driver.find_element(By.ID, "verify")
```

- ✓ Явные ожидания (Explicit Waits) реализуется с помощью инструментов `WebDriverWait` и `expected_conditions`:

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

button = WebDriverWait(driver, 5).until(
    EC.element_to_be_clickable((By.ID, "submit-button"))
)
button.click()
```

В данном примере код ожидает 5 секунд, пока кнопка не станет кликабельной

Selenium. Явные ожидания. Правила ожиданий.

```
# говорим Selenium проверять в течение 5 секунд, пока кнопка не станет кликабельной
button = WebDriverWait(browser, 5).until(
    EC.element_to_be_clickable((By.ID, "verify"))
)
button.click()
```

Правило ожидания

В модуле `expected_conditions` есть много других правил, которые позволяют реализовать необходимые ожидания:

- `title_is`
- `title_contains`
- `visibility_of_element_located`
- `visibility_of`
- `presence_of_all_elements_located`
- `text_to_be_present_in_element`
- `element_to_be_clickable`
- `element_to_be_selected`
- `element_located_to_be_selected`
- `element_selection_state_to_be`
- `element_located_selection_state_to_be`
- `alert_is_present ...`

Описание каждого правила можно найти на [сайте](#).

```
from selenium import webdriver
```

импортируем класс By, который позволяет выбрать способ поиска элемента

```
from selenium.webdriver.common.by import By
```

инициализируем драйвер браузера. После этой команды вы должны увидеть новое открытое окно браузера

```
driver = webdriver.Chrome()
```

```
driver.get("https://suninjuly.github.io/text_input_task.html")
```

команда time.sleep устанавливает паузу в 3 секунды, чтобы мы успели увидеть, что происходит в браузере

```
time.sleep(3)
```

или что бы элементы успели прогрузиться

Ищем поле для ввода текста

```
textarea = driver.find_element(By.CSS_SELECTOR, ".textarea")
```

Напишем текст ответа в найденное поле

```
textarea.send_keys("get()")
```

```
time.sleep(3)
```

Найдем кнопку, которая отправляет введенное решение

```
submit_button = driver.find_element(By.CSS_SELECTOR, ".submit-submission")
```

```
submit_button.click()
```

```
time.sleep(3)
```

```
driver.quit()
```

Пример работы с Selenium.
Здесь открываем ссылку, находим
окно ввода, вводим строку,
нажимаем на кнопку Submit.

Сохранение результатов в файл

Работа с файлами в Python

Python поддерживает множество различных типов файлов, но условно их можно разделить на два вида: **текстовые** и **бинарные**.

При работе с файлами необходимо соблюдать некоторую последовательность операций:

1. открытие файла с помощью метода **open()**
2. чтение файла с помощью метода **read()** или запись в файл посредством метода **write()**
3. закрытие файла методом **close()**

Открытие и закрытие файла

Чтобы начать работу с файлом, его надо открыть с помощью функции **open()**:

```
f = open('text.txt', 'r')
```



Первый параметр функции представляет путь к файлу. Путь файла может быть *абсолютным*, то есть начинаться с буквы диска, например, *C://somedir/somefile.txt*. Либо можно быть *относительным*, например, *somedir/somefile.txt* - в этом случае поиск файла будет идти относительно расположения запущенного скрипта Python.

```
f = open('text.txt', 'r')
```

Второй передаваемый аргумент - **mode** устанавливает режим открытия файла в зависимости от того, что мы собираемся с ним делать. Существует 4 общих режима:

- **r** (Read). Файл открывается для чтения. Если файл не найден, то генерируется исключение `FileNotFoundError`
- **w** (Write). Файл открывается для записи. Если файл отсутствует, то он создается. Если подобный файл уже есть, то он создается заново, и соответственно старые данные в нем стираются.
- **a** (Append). Файл открывается для дозаписи. Если файл отсутствует, то он создается. Если подобный файл уже есть, то данные записываются в его конец.
- **b** (Binary). Используется для работы с бинарными файлами. Применяется вместе с другими режимами - `w` или `r`.

После завершения работы с файлом его обязательно нужно закрыть методом **close()**.

Данный метод освободит все связанные с файлом используемые ресурсы.

Например:

```
somefile = open("hello.txt", "r")  
.  
.  
somefile.close()
```


При открытии файла или в процессе работы с ним можно столкнуться с различными исключениями, например, к нему нет доступа и т.д. В этом случае можно обрабатывать исключения:

```
try:
    somefile = open("hello.txt", "w")
    try:
        somefile.write("hello world")
    except Exception as e:
        print(e)
    finally:
        somefile.close()
except Exception as ex:
    print(ex)
```

В данном случае вся работа с файлом идет во вложенном блоке try. И если вдруг возникнет какое-либо исключение, то в любом случае в блоке finally файл будет закрыт.

Однако есть и более удобная конструкция - конструкция with:

```
with open("hello.txt", "w") as file_obj:
    file_obj.write("hello world")
```

Эта конструкция определяет для открытого файла переменную file_obj и выполняет набор инструкций. После их выполнения файл автоматически закрывается. Даже если при выполнении инструкций в блоке with возникнут какие-либо исключения, то файл все равно закрывается.

Какой тип файла выбрать для сохранения результатов парсинга?

Формат **CSV** (Comma Separated Values) является наиболее распространенным форматом хранения информации в удобном виде. Каждая строка в файле csv представляет отдельную запись или строку, которая состоит из отдельных столбцов, разделенных запятыми (или точкой с запятой).

```
import csv
FILENAME = "users.csv"

users = [
    ["Tom", 28],
    ["Alice", 23],
    ["Bob", 34]]

with open(FILENAME, "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(users)
```

1	date;gonkong_dollar
2	01.01.2020;0,2708
3	01.02.2020;0,27526
4	01.03.2020;0,28724
5	01.04.2020;0,33347
6	01.05.2020;0,31149
7	01.06.2020;0,31083
8	01.07.2020;0,31126

Для упрощения работы с этим форматом предоставляется специальный встроенный модуль **csv**.

Объект **writer** возвращается функцией **csv.writer(file)**.

Запись производится с помощью метода **writer.writerows(users)** Этот метод принимает набор строк. В нашем случае это двухмерный список.

Для чтения из файла нужно создать объект **reader**:

```
import csv

FILENAME = "users.csv"

with open(FILENAME, "r", newline="") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row[0], " - ", row[1])
```

Если кодировка не совпадает с ASCII, то нужно явным образом указать кодировку с помощью параметра **encoding**:

```
def save_file(items, path): 1 usage
    with open(path, 'w', newline='', encoding="utf-8") as file:
        writer = csv.writer(file, delimiter=';')
        writer.writerow(['Name', 'Price'])
```

Работа со словарями

Модуль csv имеет специальные возможности для работы со словарями.

```
import csv

FILENAME = "users2.csv"

users = [
    {"name": "Tom", "age": 28},
    {"name": "Alice", "age": 23},
    {"name": "Bob", "age": 34}
]

with open(FILENAME, "w", newline="") as file:
    columns = ["name", "age"]
    writer = csv.DictWriter(file, fieldnames=columns)
    writer.writeheader()

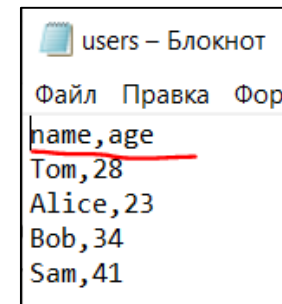
with open(FILENAME, "r", newline="") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["name"], "-", row["age"])
```

Функция **csv.DictWriter()** возвращает объект writer, который позволяет записывать в файл.

А функция **csv.DictReader()** возвращает объект reader для чтения из файла:

В этом случае каждая строка представляет собой отдельный словарь, и кроме того, производится запись и заголовков столбцов с помощью метода writeheader(), а в метод csv.DictWriter в качестве второго параметра передается набор столбцов.

При чтении строк, используя названия столбцов, можно обратиться к отдельным значениям внутри строки: row["name"].



```
def save_file(items,path):  
    with open(path, 'w', newline="",encoding="utf-8") as file:  
        writer = csv.writer(file, delimiter=';')  
        writer.writerow(['Name', 'Price'])  
        for item in items:  
            writer.writerow([item['name'], item['price']])
```

Запись файла

```
html = requests.get('https://sunlight.net/catalog').text  
soup = BeautifulSoup(html,'xml')  
items = soup.find_all('div', class_='cl-item-mobile-footer')
```

```
jewelry=[]  
for item in items:  
    soup = BeautifulSoup(html)  
    name=item.find('div', class_='cl-item-mobile-footer__subname').get_text(strip=True)  
    price = item.find('div', class_='cl-item-mobile-footer-price').find_all('span')[0].text
```

```
    jewelry.append({  
        'name': name,  
        'price': price  
    })
```

```
save_file(jewelry,"catalog.csv")
```

Анализируем результаты

Нам понадобятся две библиотеки Python:

- **NumPy** (Numerical Python) используется для обработки больших массивов данных. Он упрощает математические операции и их векторизацию на массивах.
- **Pandas** упрощает работу с табличными данными.



Что такое NumPy?



NumPy — библиотека с открытым исходным кодом для Python, реализующая множество математических операций для работы с векторами, матрицами и массивами. Важное преимущество NumPy перед собственной реализацией массивов (например на списках) - **это векторные операции, которые происходят гораздо быстрее, последовательных.**

Фактически, NumPy - это основная математическая библиотека для работы с данными (если вы решаете задачи машинного обучения или анализа данных). Именно NumPy, а не встроенный Math.

NumPy лежит в основе других важных библиотек: Pandas (работа с табличными данными), SciPy (работы с методами оптимизации и научными расчётами), Matplotlib (построение графиков) и т.д.

Арифметические операции над массивами данных

Создадим два массива NumPy и на их примере покажем преимущества пакета. Назовём массивы `data` и `ones`:

`data = np.array([1,2])`

data

1
2

`ones = np.ones(2)`

ones

1
1

Просуммировать их по позициям (т. е. просуммировать значения каждой строки) очень просто: надо ввести команду `data + ones`:

`data + ones`

=

data

1
2

+

ones

1
1

=

2
3

Сложение матриц

data + **ones** =

1	2
3	4

+

1	1
1	1

=

2	3
4	5

Скалярное произведение

data .dot(**powers_of_ten**) =

1	2	3
---	---	---

Matrix dimensions: 1x3

1	10
100	1,000
10,000	100,000

Matrix dimensions: 3x2

=

30201	302010
-------	--------

Matrix dimensions: 1x2

изменение формы матриц

data

1
2
3
4
5
6

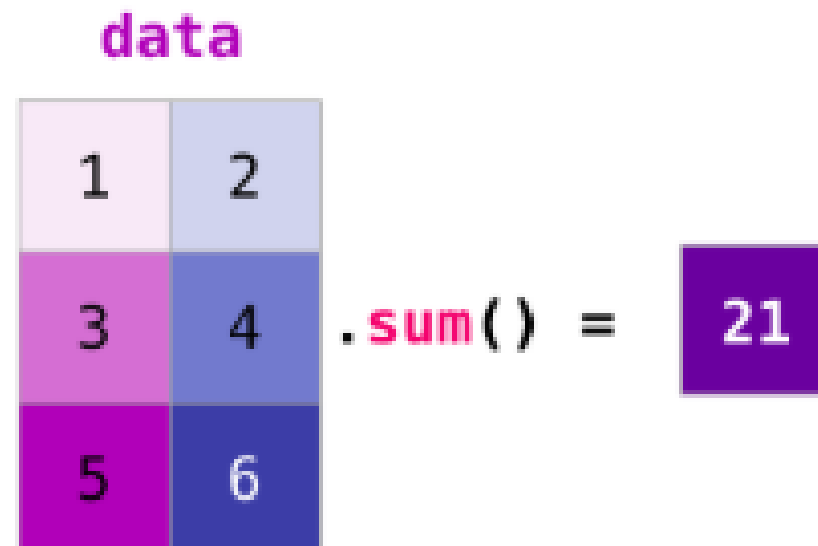
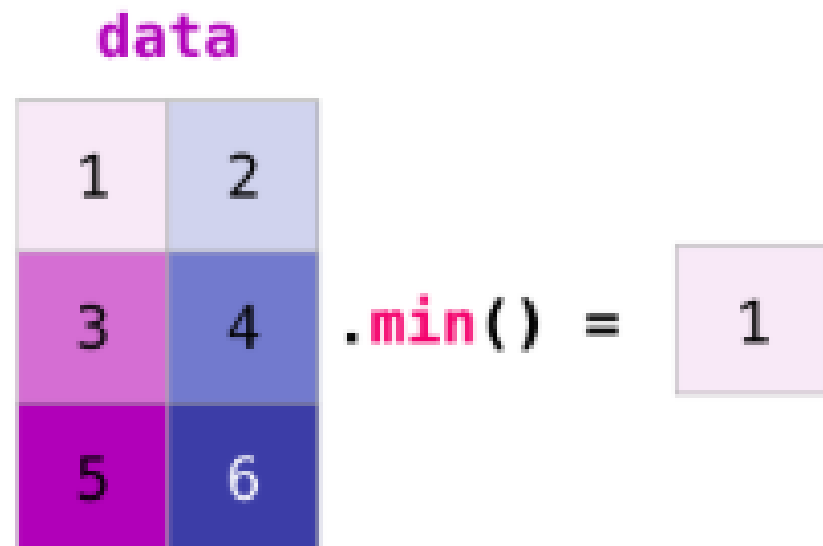
data.reshape(2,3)

1	2	3
4	5	6

data.reshape(3,2)

1	2
3	4
5	6

Агрегирование матриц



Pandas — это библиотека Python, предоставляющая широкие возможности для анализа данных.



Pandas спроектирована на основе библиотеки NumPy. Такой выбор делает pandas совместимой с большинством других модулей.

Еще одно важное решение — разработка специальных структур для анализа данных. Вместо того, чтобы использовать встроенные в Python или предоставляемые другими библиотеками структуры, были разработаны две новых **Series** и **DataFrame**.

Данные, часто хранятся в форме табличек — например, в форматах .csv, .tsv или .xlsx. С помощью библиотеки Pandas такие табличные данные очень удобно загружать, обрабатывать и анализировать.

В связке с библиотеками Matplotlib и Seaborn Pandas предоставляет широкие возможности визуального анализа табличных данных.

Структуры данных в pandas

- **Series**
- **Dataframe**

Series — это объект библиотеки pandas для представления одномерных структур данных.

Series состоит из двух связанных между собой массивов. Основной содержит данные (данные любого типа NumPy), а в дополнительном, index, хранятся метки.

Series	
index	value
0	12
1	-4
2	7
3	9

Создать структуру **Series** можно на базе различных типов данных:

- словарей Python;
- списков Python;
- массивов из numpy: ndarray;
- скалярных величин.

DataFrame — это двумерная структура данных, представляющая собой таблицу, каждый столбец которой содержит данные одного типа. Можно представлять её как словарь объектов типа Series. Структура DataFrame отлично подходит для представления реальных данных.

В отличие от Series у которого есть массив индексов с метками, ассоциированных с каждым из элементов, Dataframe имеет сразу два таких.

Первый ассоциирован со строками (рядами) и напоминает таковой из Series. Каждая метка ассоциирована со всеми значениями в ряду.

Второй содержит метки для каждой из колонок.

DataFrame			
	columns		
index	color	object	price
0	blue	ball	1.2
1	green	pen	1.0
2	yellow	pencil	0.6
3	red	paper	0.9
4	white	mug	1.7

Внутреннее представление данных разных типов в pandas
Столбцы данных группируются в блоки со значениями одинакового типа.
На рисунке пример того, как в pandas хранятся первые 12 столбцов объекта DataFrame.

DataFrame												
	date	number_of_game	day_of_week	v_name	v_league	v_game_number	h_name	h_league	h_game_number	v_score	h_score	length_outs
0	01871054	0	Thu	CL1	na	1	FW1	na	1	0	2	54.0
1	18710505	0	Fri	BS1	na	1	WS3	na	1	20	18	54.0
2	18710506	0	Sat	CL1	na	2	RC1	na	1	12	4	54.0

IntBlock

	0	1	2	3	4	5
0	01871054	0	1	1	0	2
1	18710505	0	1	1	20	18
2	18710506	0	2	1	12	4

ObjectBlock

	0	1	2	3	4
0	Thu	CL1	na	FW1	na
1	Fri	BS1	na	WS3	na
2	Sat	CL1	na	RC1	na

FloatBlock

	0
0	54.0
1	54.0
2	54.0

```
In [ ]: import pandas as pd
import numpy as np
```

```
In [6]: ave_data = np.array(np.arange(24)).reshape((6,4))
dates = pd.date_range('20210101', periods=6)
ave_df = pd.DataFrame(ave_data, index=dates, columns=list('1234'))
ave_df
```

Out[6]:

	1	2	3	4
2021-01-01	0	1	2	3
2021-01-02	4	5	6	7
2021-01-03	8	9	10	11
2021-01-04	12	13	14	15
2021-01-05	16	17	18	19
2021-01-06	20	21	22	23

Getting started **User Guide** API reference Development Release notes

Q Search the docs ...

10 minutes to pandas

Intro to data structures

Essential basic functionality

IO tools (text, CSV, HDF5, ...)

Indexing and selecting data

MultiIndex / advanced indexing

Merge, join, concatenate and compare

Reshaping and pivot tables

Working with text data

Working with missing data

Duplicate Labels

Categorical data

Null and integer data types

10 minutes to pandas

This is a short introduction to pandas, geared mainly for new users. You can see more complex recipes in the [Cookbook](#).

Customarily, we import as follows:

```
In [1]: import numpy as np
In [2]: import pandas as pd
```

Object creation

See the [Data Structure Intro](#) section.

Creating a **Series** by passing a list of values, letting pandas create a default integer index:

Пример на датасете Титаник

```
In [1]: import pandas as pd
```

```
In [2]: titanic = pd.read_csv("data/titanic.csv")
```

Как рассчитать сводную статистику?

Агрегированная статистика

Какой средний возраст пассажиров Титаника?

```
In [6]: titanic["Age"].mean()
```

```
Out[6]: 29.69911764705882
```

Каков средний возраст и стоимость билета пассажиров «Титаника»?

```
In [7]: titanic[["Age", "Fare"]].median()
```

```
Out[7]: Age      28.0000  
Fare      14.4542  
dtype: float64
```

Функция `pandas.DataFrame.describe` рассчитывает параметры описательной статистики

In [8]: `titanic[["Age", "Fare"]].describe()`

Out[8]:

	Age	Fare
count	714.000000	891.000000
mean	29.699118	32.204208
std	14.526497	49.693429
min	0.420000	0.000000
25%	20.125000	7.910400
50%	28.000000	14.454200
75%	38.000000	31.000000
max	80.000000	512.329200

Вместо predefined статистики можно определить конкретные комбинации агрегированной статистики для заданных столбцов с помощью `DataFrame.agg()` метода:

[10]: `titanic.agg(
 {
 "Age": ["min", "max", "median"],
 "Fare": ["min", "max", "median", "mean"],
 }
)`

[10]:

	Age	Fare
min	0.42	0.000000
max	80.00	512.329200
median	28.00	14.454200

Каков средний возраст пассажиров Титаника мужчинами и женщинами?

```
In [11]: titanic[["Sex", "Age"]].groupby("Sex").mean()
```

```
Out[11]:
```

	Age
Sex	
female	27.915709
male	30.726645

Если не указывать столбцы, то mean-метод применяется к каждому столбцу, содержащему числовые данные:

```
In [12]: titanic.groupby("Sex").mean()
```

```
Out[12]:
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
Sex							
female	431.028662	0.742038	2.159236	27.915709	0.694268	0.649682	44.479818
male	454.147314	0.188908	2.389948	30.726645	0.429809	0.235702	25.523893

Шаг 5: Визуализируем результаты



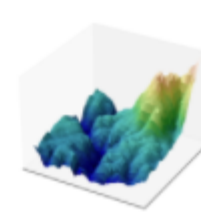
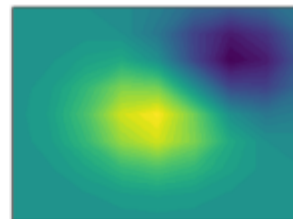
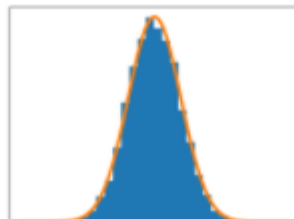
<https://matplotlib.org/>

Монтаж Документация Примеры Учебники Содействие

[домой](#) | [содержание](#) » [Matplotlib: построение графиков Python](#)

Matplotlib: Визуализация с помощью Python

Matplotlib - это комплексная библиотека для создания статических, анимированных и интерактивных визуализаций в Python.



Matplotlib делает легкие вещи легкими, а сложные - возможными.

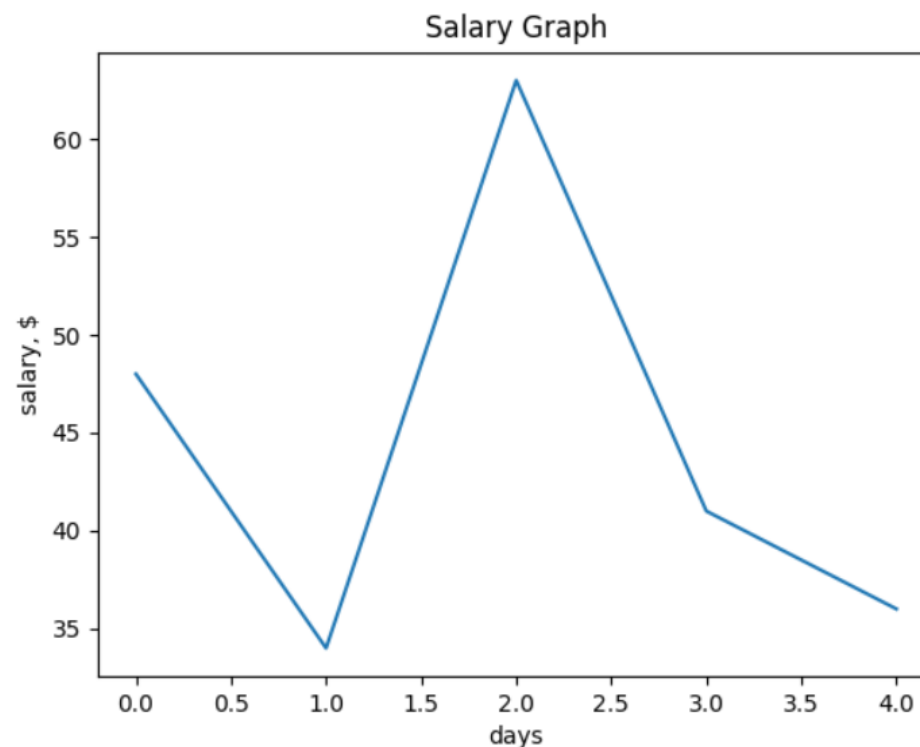
Функция **plot()** является универсальной функцией и принимает произвольное количество аргументов.

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4]) ← y
plt.ylabel('some numbers')
plt.show()
```

x y

```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
```

```
1  import matplotlib.pyplot as plt
2
3  x_list=list(range(0,5))
4  y_list= [48,34,63,41,36]
5
6  plt.title('Salary Graph')
7  plt.xlabel('days')
8  plt.ylabel('salary, $')
9  plt.plot(x_list,y_list)
10
11 plt.show()
```



Маркеры

```
import matplotlib.pyplot as plt

x_list=list(range(0,5))
y_list= [48,34,63,41,36]

plt.title('Salary Graph')
plt.xlabel('days')
plt.ylabel('salary, $')
plt.plot(x_list,y_list,marker='o')

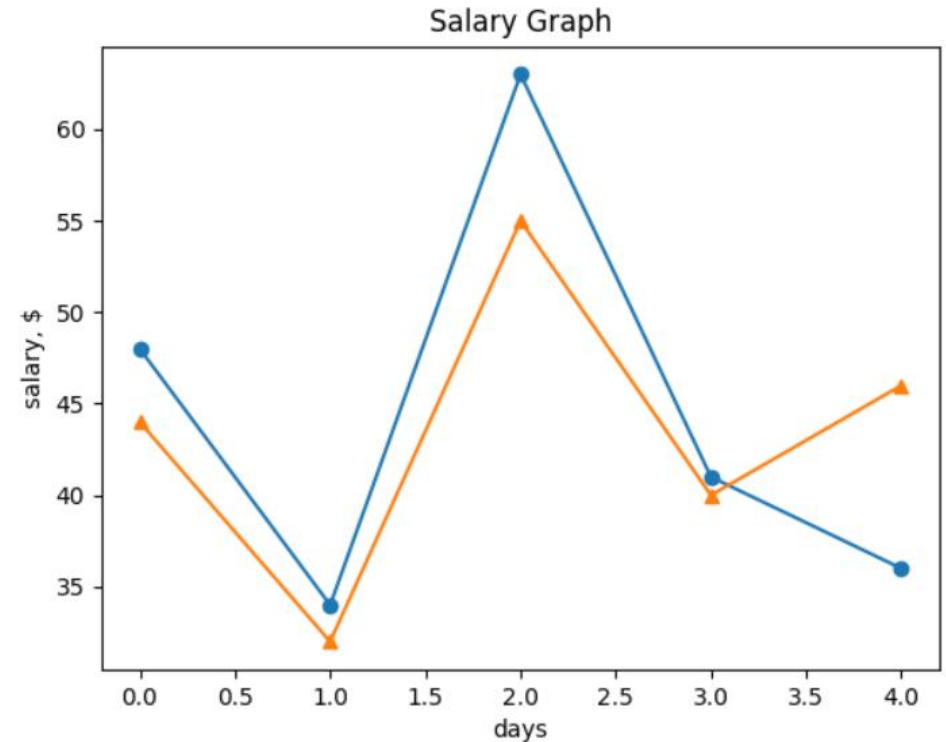
plt.show()
```

marker	symbol	description
"."	•	point
","	.	pixel
"o"	●	circle
"v"	▼	triangle_down
"^"	▲	triangle_up
"<"	◀	triangle_left
">"	▶	triangle_right
"1"	⋈	tri_down
"2"	⋈	tri_up
"3"	↰	tri_left
"4"	↱	tri_right
"8"	●	octagon
"s"	■	square
"p"	⬠	pentagon
"p"	⊕	plus (filled)
"*"	★	star

https://matplotlib.org/stable/api/markers_api.html

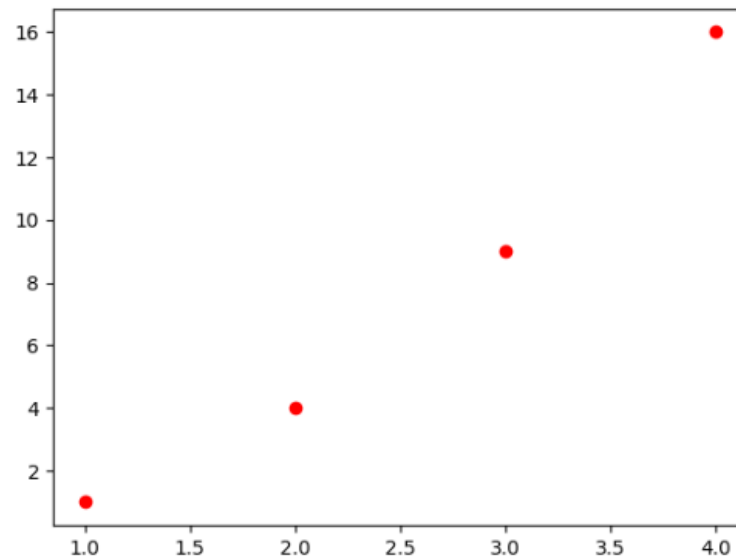
Два графика в одних осях

```
1 import matplotlib.pyplot as plt
2
3 x_list=list(range(0,5))
4 y1_list= [48,34,63,41,36]
5 y2_list= [44,32,55,40,46]
6
7 plt.title('Salary Graph')
8 plt.xlabel('days')
9 plt.ylabel('salary, $')
10 plt.plot(x_list,y1_list,marker='o')
11 plt.plot(x_list,y2_list,marker='^')
12 plt.show()
```



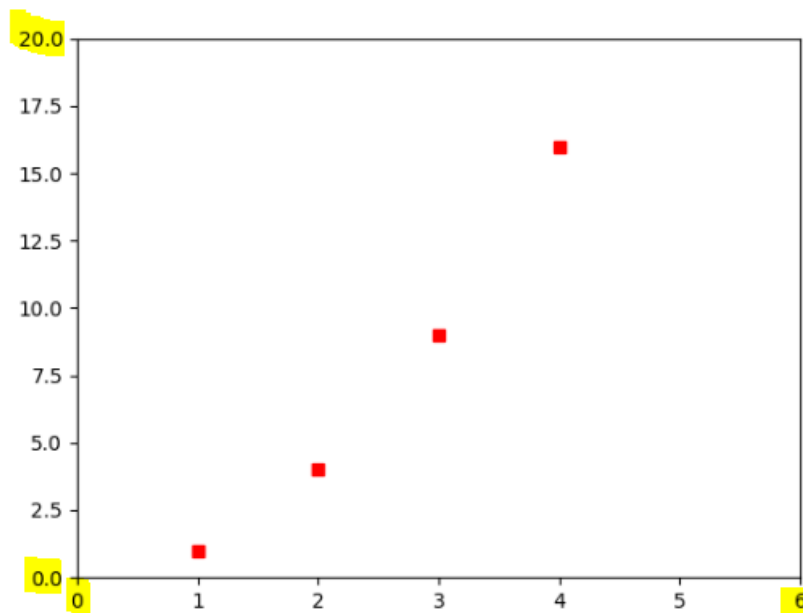

```
>>> plot(x, y)           # plot x and y using default line style and color
>>> plot(x, y, 'bo')     # plot x and y using blue circle markers
>>> plot(y)              # plot y using x as index array 0..N-1
>>> plot(y, 'r+')        # ditto, but with red plusses
```

```
1 import matplotlib.pyplot as plt
2
3 plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'ro')
4
5 plt.show()
```



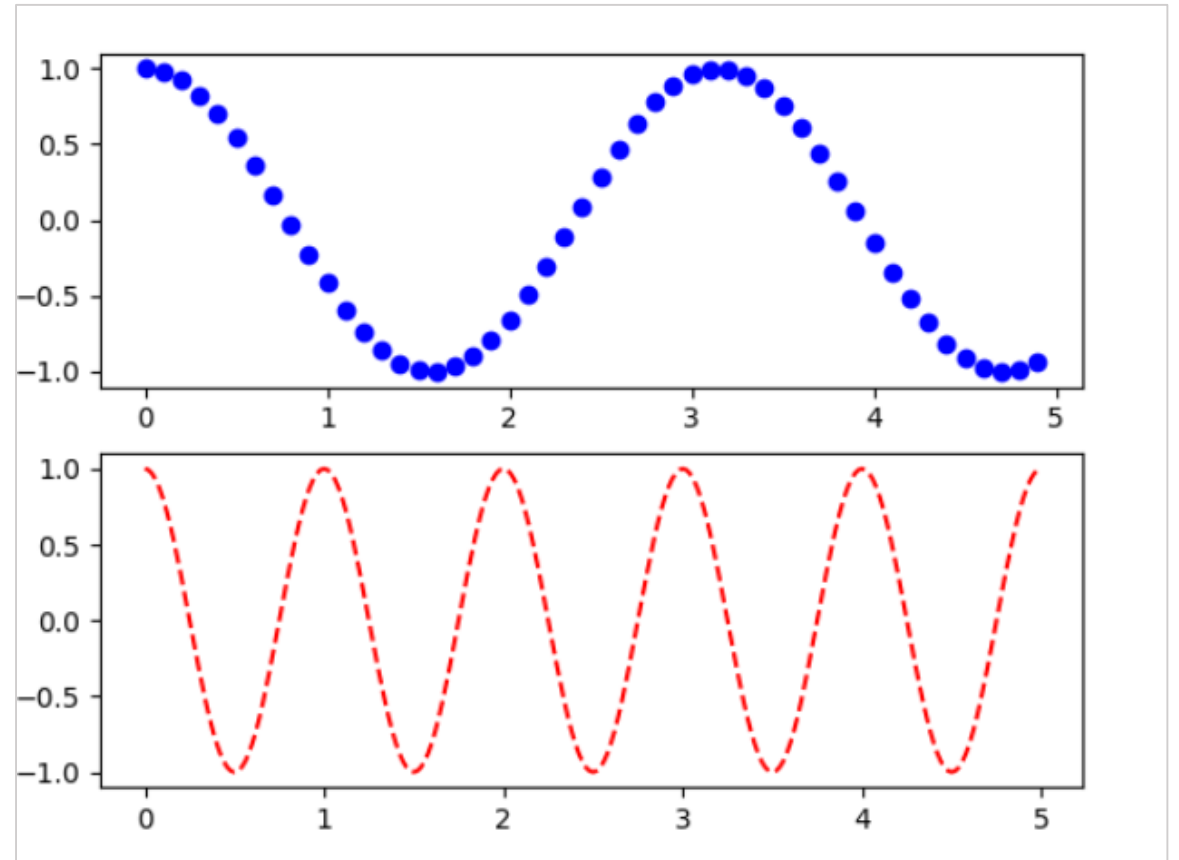
Форматирование диапазона осей

```
1 import matplotlib.pyplot as plt
2
3 plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'rs')
4 plt.axis([0, 6, 0, 20])
5
6 plt.show()
```



Размещение графиков

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(t):
5     return np.cos(2*np.pi*t2)
6
7 t1 = np.arange(0.0, 5.0, 0.1)
8 t2 = np.arange(0.0, 5.0, 0.02)
9
10 plt.figure()
11 plt.subplot(211)
12 plt.plot(t1, np.cos(2*t1), 'bo')
13
14 plt.subplot(212)
15 plt.plot(t2, f(t2), 'r--')
16 plt.show()
```



Параметры **subplot()**:

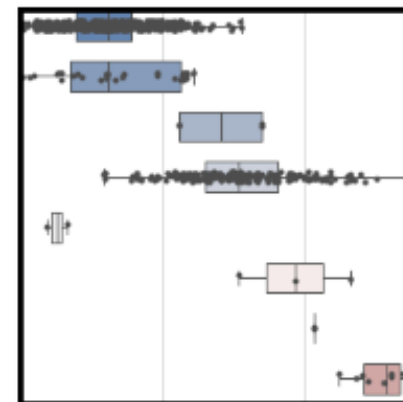
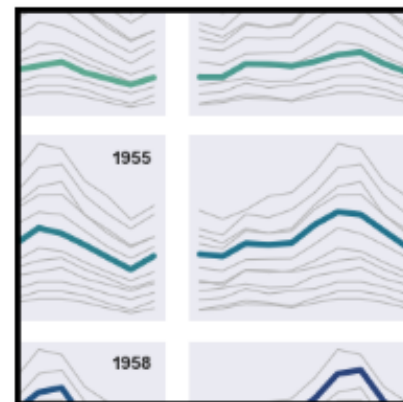
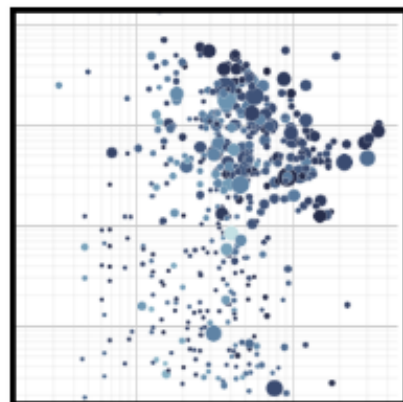
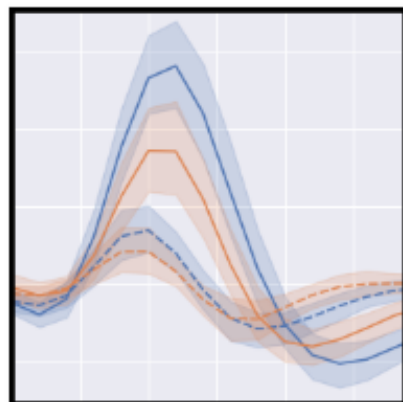
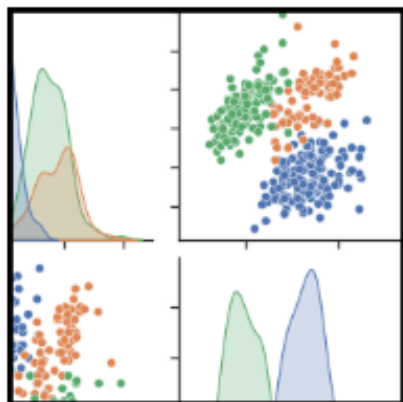
1. количество строк;
2. количество столбцов
3. индекс ячейки.

Seaborn — это библиотека для создания статистических графиков на Python. Она основывается на matplotlib и тесно взаимодействует со структурами данных pandas.



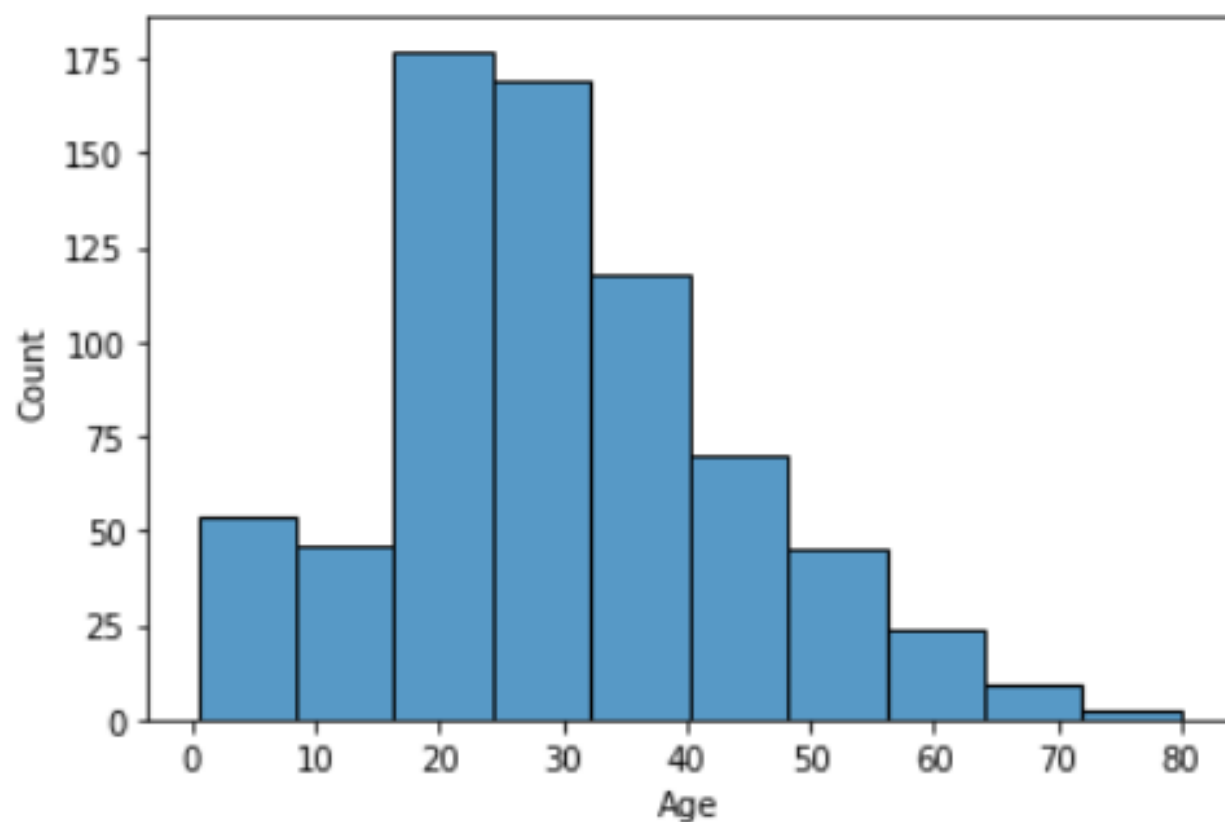
[Installing](#) [Gallery](#) [Tutorial](#) [API](#) [Releases](#) [Citing](#) [FAQ](#)

seaborn: statistical data visualization



```
In [4]: import seaborn as sns
sns.histplot(data=titanic["Age"],bins=10)
```

Out[4]: <AxesSubplot:xlabel='Age', ylabel='Count'>



```
In [36]: s.describe()
```

```
Out[36]: count    20.000000  
mean      7.250000  
std       4.191156  
min       1.000000  
25%       4.750000 1  
50%       6.000000 2  
75%      10.000000 3  
max      18.000000  
dtype: float64
```

```
In [34]: s.median()
```

```
Out[34]: 6.0
```

```
In [48]: sns.boxplot(x=s)
```

```
Out[48]: <AxesSubplot:>
```

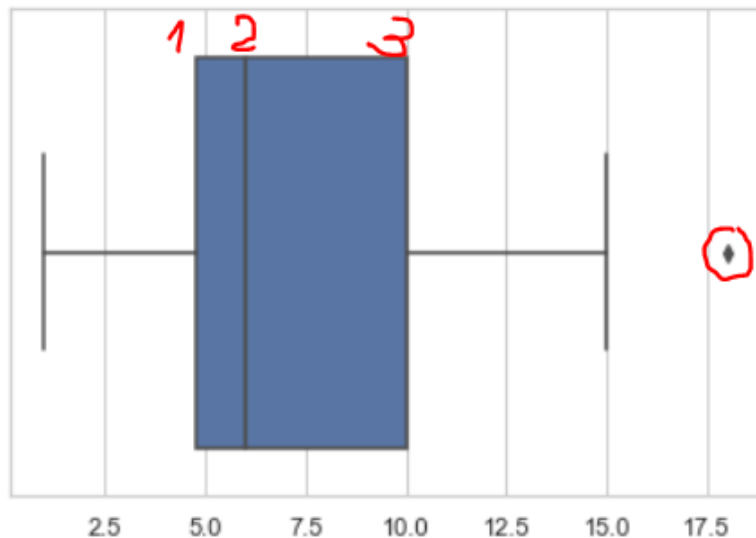


График box-plot

Центром ящика является медиана наших данных или второй квартиль, верхняя граница = 3-й квартиль, а нижняя граница = 1-й квартиль.

Почему некоторые точки на графике отображены отдельно?

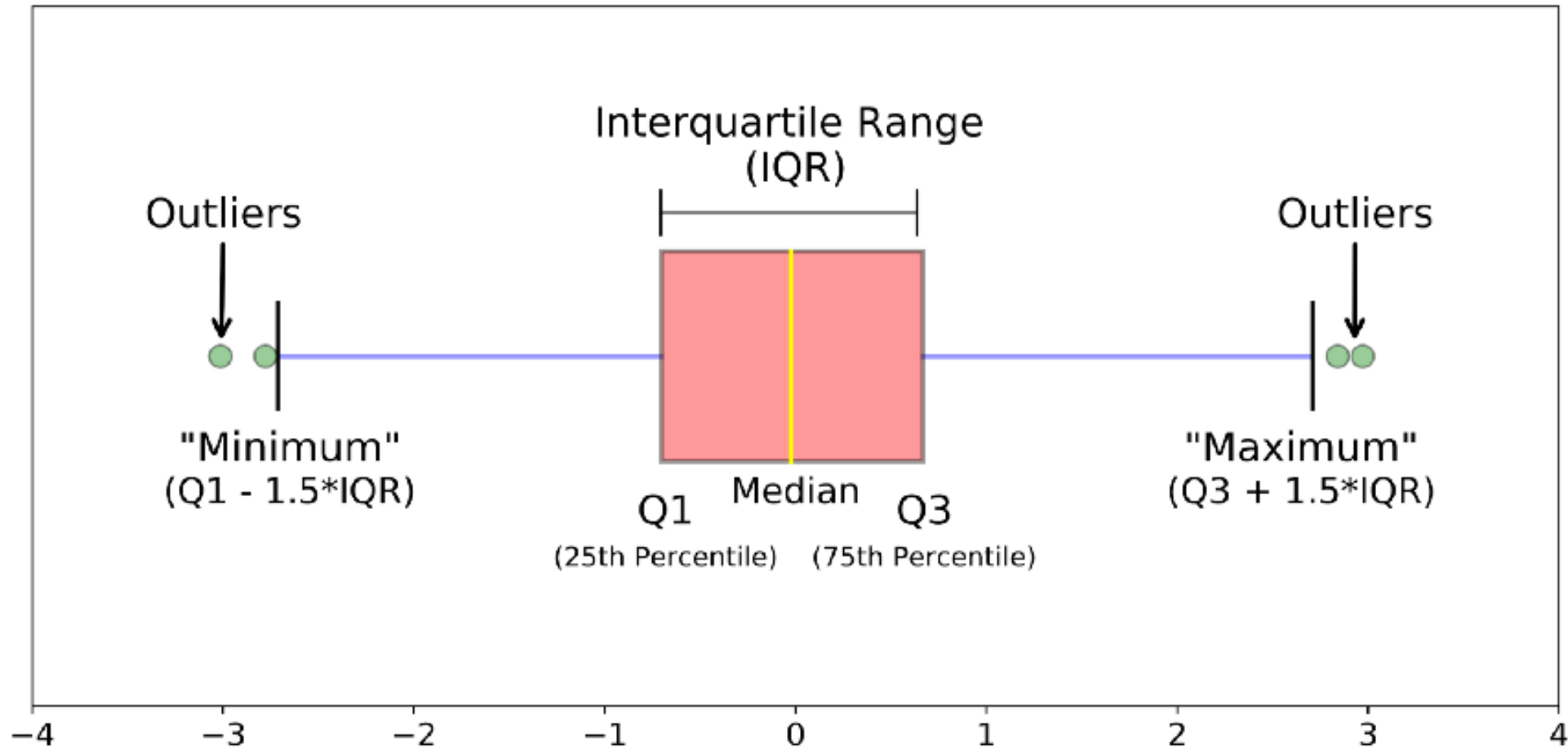
Если мы посчитаем разность между 3-м и 1-м квартилем - это межквартильный размах (мера изменчивости).

Чем выше межквартильный размах, тем больше вариативность нашего признака.

Отложим мысленно 1,5 межквартильного размаха вверх и вниз от 1-го и 3-го квартилей. Те значения признака, которые последними принадлежат этому промежутку и будут границами усов.

Точки, которые превосходят полтора межквартильного размаха - наносятся на график отдельно.

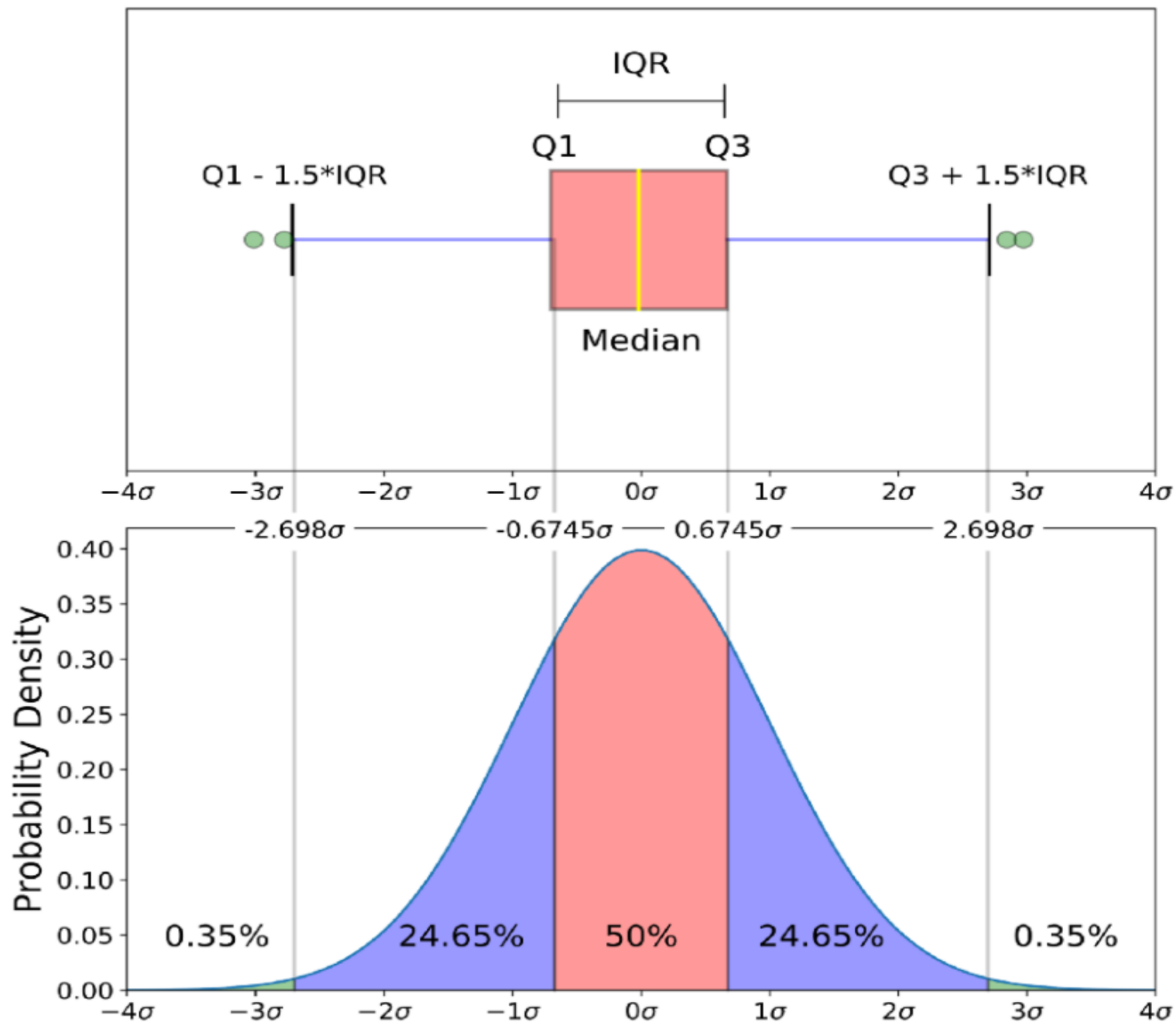
box plot Ящик с усами Диаграмма размаха



IQR - размах

Q1 – 25 перцентиль (1й квартиль)

Q3 – 75 перцентиль (3й квартиль)

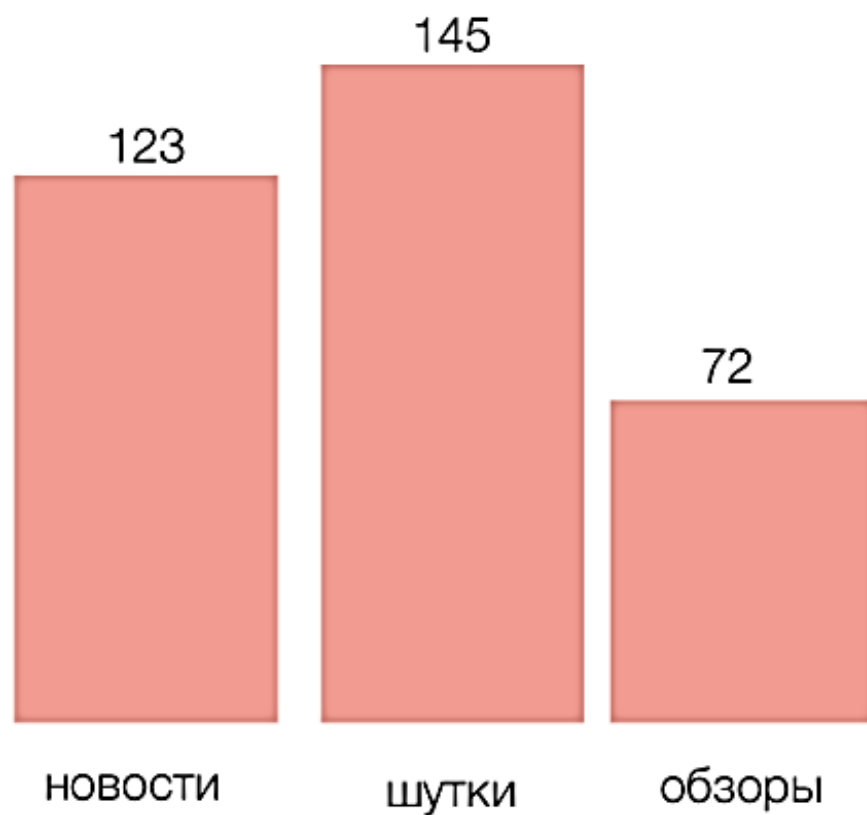


Box-plot наглядно визуализирует распределение случайной величины.

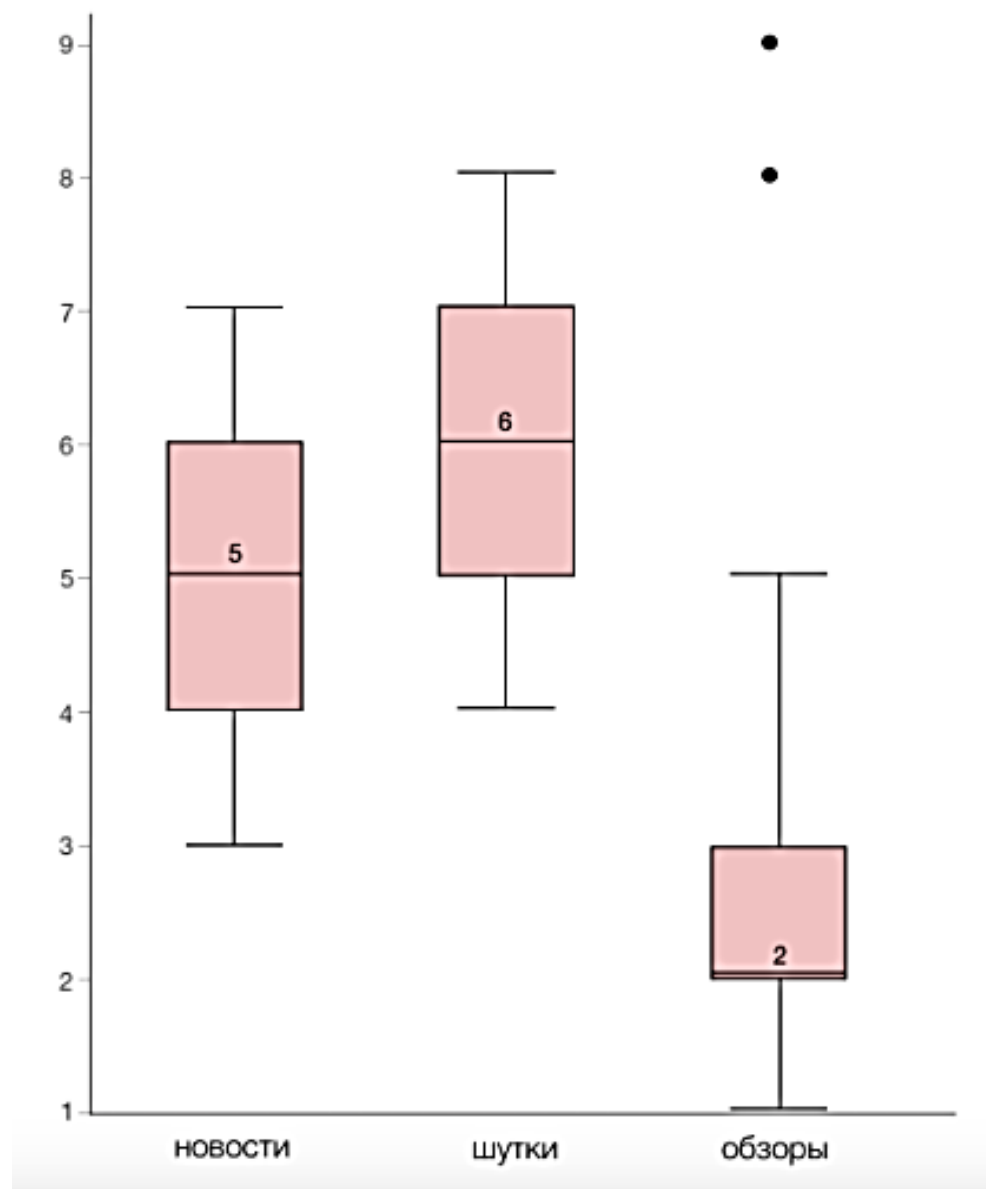
Из рисунка видно, что ящик содержит 50 процентов всех данных, а за пределами усов содержится 0,35 процентов, т.е. очень маленькое количество данных, которые можно расценить как выбросы.

Пример: анализ эффективных постов блога

Сумма лайков по категориям



Распределение лайков по категориям



Парсить законно?

Сам по себе сбор данных из открытых источников законом не запрещен. НО искать, и использовать информацию нужно с соблюдением законодательства — и тут в силу вступают другие правовые нормы:

- ✓ **Если при помощи парсеров вы полностью копируете информацию с сайтов конкурентов на собственный ресурс**, это может привести к нарушению интеллектуального права.
- ✓ **Чрезмерно агрессивный парсер может создать большую нагрузку на целевой сайт, которая будет выглядеть как DDOS-атака.** Если вы парсите такой программой интернет-магазин, то он может стать недоступным на несколько часов, и владельцы сайта потерпят убытки. Или у владельцев могут возрасти затраты на обслуживание серверов.
- ✓ **В Уголовном кодексе предусмотрена ответственность за «неправомерный доступ к охраняемой законом информации».** Эта формулировка включает в себя персональные данные или коммерческую тайну.
- ✓ Согласно Закону о персональных данных, для сбора и использования даже находящихся в открытом доступе персональных данных нужно получить согласие пользователя. Строго говоря, **сбирать данные пользователей для запуска таргетированной рекламы — тоже незаконно.** Но установить факт парсинга данных при запуске рекламы сейчас технически невозможно, поэтому многие компании продолжают использовать этот инструмент.

Вывод: парсить можно, главное, чтобы этот процесс не приводил к случаям, когда может возникнуть дополнительная ответственность.

Для каких целей можно использовать парсинг

- ✓ **Исследование рынка.** Парсинг позволяет быстро оценить, какие товары и цены у конкурентов.
- ✓ **Анализ динамики изменений.** Парсинг можно проводить регулярно, чтобы оценивать, как менялись какие-то показатели. Например, росли или падали цены, изменялось количество онлайн-объявлений или сообщений на форуме.
- ✓ **Наполнение каталога интернет-магазина.** Чтобы упростить этот процесс, часто парсят зарубежные магазины и просто переводят информацию о товарах.
- ✓ **Составление клиентской базы.** В этом случае парсят контактные данные, например, пользователей соцсетей, участников форумов и т. д. Но стоит помнить, что сбор информации, которой нет в открытом доступе, незаконен.
- ✓ **Сбор отзывов и комментариев на форумах, в соцсетях.**
- ✓ **Создание контента, который строится на выборке данных.** Например, результаты спортивных состязаний, инфографики по изменению цен, погоды и т. д.
- ✓ **Поиск внешних ссылок, ведущих на вашу площадку.** Это поможет оценить работу подрядчика по линкбилдингу. Как проверять внешние ссылки и какими инструментами это делать, подробно описано в этой [статье](#).